

NATIONAL UNIVERSITY

551 M.F. JHOCSON ST., SAMPALOC, MANILA

COLLEGE OF ENGINEERING

Name/Section	SANTOS, KERVEIN KYLE P. COE2322	Rating
Date performed	09/11/2026	
Date submitted	09/11/2026	

SOFTWARE DESIGN EXPERIMENT NO. 7

Minimum-Change Approach using Flask + HTML/CSS/JavaScript Campus Hardware Inventory System

1. Laboratory Objective

Convert your already-working Python/Tkinter desktop application into a browser-based Flask application without throwing away your original work. The objective is to reuse the existing database, authentication, validation, inventory, borrowing, return, and administrative logic as much as possible, while replacing the desktop interface with a web interface.

- Preserve the original Python/database/business logic as much as possible.
- Identify Tkinter-only code that cannot run in a normal web browser.
- Comment or isolate desktop GUI code instead of deleting your original work.
- Create a Flask web bridge that calls the existing Python controllers.
- Replace Tkinter windows/widgets with HTML pages and browser forms.
- Use CSS for presentation and JavaScript only for small browser-side interactions.
- Run and test the converted application locally in Google Chrome.

2. IMPORTANT: What Changes and What Does NOT Change

This is the most important rule of the experiment. You are adapting the interface and communication method; you are NOT rebuilding the entire system.

System Part / Feature	CHANGE?	Student Instruction
Original Laboratorysystem.py / supplementary_auth.py	NO	Keep your original file. Do not throw away working controller/database code.
SQLite database and existing records	NO	Keep and back up hardware_inventory.db. Use the same local database for Lab 1.
init_db()	NO	Reuse it. Flask may call it when the web application starts.
AuthController	NO	Reuse login, registration, reset, password and lockout logic.
InventoryController	NO / MINIMAL	Reuse existing inventory, history and return logic. Only add backend support when the new quantity/borrow-approval requirement is not already supported.
Pydantic validation	NO	Keep existing validation.

bcrypt authentication	NO	Keep existing password hashing/authentication.
Inventory status rules	NO	Keep the existing In Stock / Low Stock / Out of Stock rules.
Borrow workflow	YES	Web version uses Request Borrow → Admin Approval → Borrowed.
Borrow quantity	YES	Student specifies quantity to borrow; maximum is available stock.
Return workflow	YES / PRESERVED	Request Return → Admin Approval → Restocked. Rejecting a return keeps the item borrowed.
Tkinter AuthView	NO — isolate	Do not use it in Flask. Keep it for the original desktop version.
Tkinter InventoryView	NO — isolate	Do not use it in Flask. Keep it for comparison.
start_app() / root.mainloop()	NO — do not call	Flask starts the web server instead.
app.py	YES — NEW	Create the thin Flask bridge between browser and existing Python controllers.
HTML templates	YES — NEW	Create/modify login, register, reset and NICE dashboard pages.
CSS	YES — NEW / WEB UI	Use the NICE UI design supplied for the web version.
JavaScript	YES — SMALL INTERACTIONS	Use for selection controls, confirmations, quantity input limits and similar browser interactions only.
Total Asset Valuation	YES — REMOVE	Do not display it in the current dashboard.
Visible Items	YES — RENAME/REMOVE	Use Total Stocks instead.
Active Loans (visible wording)	YES — RENAME	Use Borrowed Items / My Borrowed Items.
Total Stocks	YES — ADD	Show the total inventory quantity.
Admin dashboard	YES — UPDATE	Show Borrowed Items, Pending Borrow Requests, Pending Returns, Password Reset Requests, Total Stocks and Account Role.
User dashboard	YES — UPDATE	Show Borrowed Items, Pending Borrow Requests, Total Stocks and Account Role.

3. Learning Outcomes

- Explain the difference between a desktop GUI and a web interface.
- Identify reusable Python/database/controller code.
- Identify Tkinter-specific code that cannot simply be rendered by a browser.
- Create a Flask adapter that communicates with existing Python controllers.
- Create Jinja HTML templates and forms.
- Use CSS to create a consistent web interface.
- Use JavaScript for limited browser-side interaction without duplicating business logic.
- Implement and test quantity-based borrowing and administrator approval.
- Test authentication lockout, inventory, borrowing, returns and history.
- Explain why local SQLite is suitable for this exercise but is not the preferred persistent database for public deployment.

4. Starting Application

Use your completed Campus Hardware Inventory desktop application as the starting point. The original program contains database initialization, AuthController, InventoryController, validation, bcrypt authentication, SQLite operations, Tkinter views and a desktop entry point.

KEEP / REUSE	ISOLATE / DO NOT CALL	ADD FOR WEB
init_db()	AuthView	app.py
SQLite structure/data	InventoryView	HTML/Jinja templates
AuthController	start_app()	CSS / static/style.css
InventoryController	tk.Tk() / mainloop()	Browser forms and routes
Pydantic + bcrypt	Tkinter widgets	Web testing
Business rules	messagebox/ttk for web UI	Borrow quantity + approval UI

5. Why Tkinter Is Not Rewritten as HTML

A browser cannot directly display a Tkinter Label, Entry, Button, Treeview or Frame. These are desktop GUI objects. The web browser understands HTML, CSS and JavaScript. Flask receives browser requests and calls Python.

Original Desktop	Web Replacement	Purpose
Tkinter Label	HTML <label>, <h1>, <div>	Display text
Tkinter Entry	HTML <input>	Receive text/values
Tkinter Button	HTML <button> / form submit	Trigger an action
Tkinter Treeview	HTML <table>	Display records
Tkinter Frame	HTML <div> / <section>	Group content
messagebox.showinfo	Flash/HTML message	Show feedback
messagebox.askyesno	JavaScript confirm()	Ask for confirmation
root.mainloop()	Flask web server	Run the application
Tkinter window	Web page	User interface
Tkinter layout	HTML + CSS	Presentation/layout
Tkinter events	HTML forms + Flask routes + JS	Browser interaction

6. Step-by-Step Procedure

Step 1 — Back Up Your Original Project

Copy the entire working desktop project. Also back up hardware_inventory.db. Do not experiment on the only copy of your database.

Step 2 — Create a Web Conversion Folder

Create a new folder, for example LaboratorySystem_Web_Lab1. Copy the original Python source, the database if using an existing local database, and the files required by your project.

Step 3 — Keep the Original Python Source

Do not rename the original source into app.py. Keep it recognizable as the original student work. The web version will import reusable functions/classes from it.

Step 4 — Identify the Reusable Code

Locate init_db(), AuthController and InventoryController. These are the core pieces that the web application should reuse.

Step 5 — Isolate the Tkinter Interface

Do NOT delete AuthView or InventoryView. Add a clear comment such as '# DESKTOP ONLY — retained for the original desktop version'. The Flask version must not call start_app(). Because the desktop start block is protected by if __name__ == '__main__', importing the module does not automatically launch Tkinter.

Step 6 — Create the Python Environment

Open PowerShell in the project folder and create/activate an environment. Example: python -m venv .venv, then .venv\Scripts\Activate.ps1.

Step 7 — Install Requirements

Run `python -m pip install -r requirements.txt`. The web project uses Flask, while bcrypt and Pydantic are already required by the original system. Gunicorn is included for later production deployment.

Step 8 — Create app.py

Create the Flask bridge. Import Flask and the reusable `init_db()`, `AuthController` and `InventoryController`. Add routes for login, register, reset request, dashboard, inventory actions, borrow actions, return actions, admin approvals and logout as required by the supplied web design.

```
# =====
# CAMPUS HARDWARE INVENTORY SYSTEM
# Flask Web Application
# =====
#
# This file connects the web browser to the existing
# Laboratorysystem.py.
#
# The original database and controller functions are reused.
# =====

# =====
# 1. IMPORT FLASK TOOLS
# =====
from flask import (
    Flask,
    render_template,
    request,
    redirect,
    url_for,
    session,
    flash,
    send_file
)

# Used to create login/admin protection decorators.
from functools import wraps

# Used for environment variables and file paths.
import os

# =====
# 2. IMPORT THE ORIGINAL PYTHON SYSTEM
# =====

# We reuse the existing database and controllers.
from Laboratorysystem import (
    init_db,
    AuthController,
    InventoryController
)

# =====
# 3. CREATE THE FLASK APPLICATION
# =====

app = Flask(__name__)
```

```

# Secret key is required for Flask sessions.
app.secret_key = os.environ.get(
    "SECRET_KEY",
    "lab1-development-secret-change-me"
)
# =====
# 4. LOGIN REQUIRED DECORATOR
# =====
#
# This checks if the user is logged in before accessing
# protected pages.
# =====
def login_required(view):

    @wraps(view)
    def wrapped(*args, **kwargs):

        # If there is no username in the session,
        # the user is not logged in.
        if "username" not in session:

            flash(
                "Please log in first.",
                "warning"
            )

            return redirect(
                url_for("login")
            )

        return view(*args, **kwargs)

    return wrapped
# =====
# 5. ADMIN REQUIRED DECORATOR
# =====
#
# This allows only ADMIN accounts to access admin functions.
# =====
def admin_required(view):

    @wraps(view)
    def wrapped(*args, **kwargs):

        # Check the user's role.
        if session.get("role") != "ADMIN":

            flash(
                "Administrator access required.",
                "danger"
            )

            return redirect(
                url_for("dashboard")
            )

        return view(*args, **kwargs)

    return wrapped

```

```

# =====
# 6. HOME / INDEX ROUTE
# =====
#
# "/" is the starting address of the web application.
# =====
@app.route("/")
def index():

    # If the user is already logged in,
    # go to the dashboard.
    if "username" in session:

        return redirect(
            url_for("dashboard")
        )

    # Otherwise, show the login page.
    return redirect(
        url_for("login")
    )
# =====
# 7. LOGIN ROUTE
# =====

@app.route("/login", methods=["GET", "POST"])
def login():

    # GET = display the login page.
    # POST = process the login form.
    if request.method == "POST":

        # Get the values entered by the user.
        username = request.form.get(
            "username",
            ""
        ).strip()

        password = request.form.get(
            "password",
            ""
        ).strip()

        # Make sure both fields have values.
        if not username or not password:

            flash(
                "Username and password are required.",
                "danger"
            )

            return render_template(
                "login.html"
            )

        # Use the existing authentication controller.
        ok, msg, role, is_locked, email = (
            AuthController.login_user(
                username,
                password
            )

```

```

)

# Login successful.
if ok:

    # Remove any previous session data.
    session.clear()

    # Save the user's information in the session.
    session["username"] = username
    session["role"] = role
    session["email"] = email

    flash(
        msg,
        "success"
    )

    return redirect(
        url_for("dashboard")
    )

# Login failed.
flash(
    msg,
    "danger"
)

# Return to the login page.
return render_template(
    "login.html",
    locked=is_locked,
    locked_username=username
)

# Display the login page.
return render_template(
    "login.html"
)

# =====
# 8. REGISTRATION ROUTE
# =====
@app.route("/register", methods=["GET", "POST"])
def register():

    # GET = display registration page.
    if request.method == "GET":

        return render_template(
            "register.html"
        )

    # Get registration information.
    username = request.form.get(
        "username",
        ""
    ).strip()

    email = request.form.get(
        "email",
        ""

```

```

).strip()

password = request.form.get(
    "password",
    ""
).strip()

role = request.form.get(
    "role",
    "USER"
).strip().upper()

# Only USER and ADMIN are accepted.
if role not in ("USER", "ADMIN"):

    role = "USER"

# Check required fields.
if not username or not email or not password:

    flash(
        "All registration fields are required.",
        "danger"
    )

    return redirect(
        url_for("login")
    )

# Use the original registration function.
ok, msg = AuthController.register_user(
    username,
    email,
    password,
    role=role
)

# Show the result.
flash(
    msg,
    "success" if ok else "warning"
)

return redirect(
    url_for("login")
)

# =====
# 9. PASSWORD RESET REQUEST
# =====
@app.route(
    "/reset-request",
    methods=["GET", "POST"]
)
def reset_request():

    # GET = display reset page.
    if request.method == "GET":

        return render_template(
            "reset.html"
        )

```

```

# Get reset information.
username = request.form.get(
    "username",
    ""
).strip()

email = request.form.get(
    "email",
    ""
).strip()

new_password = request.form.get(
    "new_password",
    ""
).strip()

confirm_password = request.form.get(
    "confirm_password",
    ""
).strip()

# Check required fields.
if (
    not username
    or not email
    or not new_password
    or not confirm_password
):

    flash(
        "All reset fields are required.",
        "danger"
    )

    return redirect(
        url_for("login")
    )

# Check if passwords match.
if new_password != confirm_password:

    flash(
        "New passwords do not match.",
        "danger"
    )

    return redirect(
        url_for("login")
    )

# Use the original password reset function.
ok, msg = (
    AuthController.submit_password_reset_request(
        username,
        email,
        new_password
    )
)

flash(

```

```

        msg,
        "success" if ok else "danger"
    )

    return redirect(
        url_for("login")
    )
# =====
# 10. DASHBOARD ROUTE
# =====
@app.route("/dashboard")
@login_required
def dashboard():

    # Get search text from the browser.
    search = request.args.get(
        "search",
        ""
    ).strip()

    # Get selected category.
    category = request.args.get(
        "category",
        "ALL"
    )

    # Use ALL when no category was selected.
    if not category:

        category = "ALL"

    # Get inventory based on search/filter.
    items = InventoryController.get_all_items(
        search_text=search,
        category=category
    )

    # Get all available categories.
    categories = InventoryController.get_categories()
# =====
# TOTAL STOCKS
# =====
#
# Get ALL inventory records so that Total Stocks
# is not affected by the search box or category filter.
# =====
all_items_for_total = (
    InventoryController.get_all_items(
        search_text="",
        category="ALL"
    )
)

# Add all stock quantities.
total_stocks = sum(
    item[3]
    for item in all_items_for_total
)
# =====
# CREATE EMPTY DATA LISTS
# =====

```

```

active_loans = []
history = []
pending_returns = []
pending_borrows = []
pending_borrow_requests = []
all_loans = []
pending_resets = []
# =====
# USER DATA
# =====
if session["role"] == "USER":

    # Items currently borrowed by the user.
    active_loans = (
        InventoryController.get_user_active_loans(
            session["username"]
        )
    )

    # Borrow requests waiting for admin approval.
    pending_borrow_requests = (
        InventoryController.get_user_pending_borrows(
            session["username"]
        )
    )

    # User's borrowing history.
    history = (
        InventoryController.get_user_loan_history(
            session["username"]
        )
    )
# =====
# ADMIN DATA
# =====
else:

    # Returns waiting for admin approval.
    pending_returns = (
        InventoryController.get_pending_returns()
    )

    # Borrow requests waiting for approval.
    pending_borrows = (
        InventoryController.get_pending_borrows()
    )

    # Complete borrowing history.
    all_loans = (
        InventoryController.get_all_loans_history()
    )

    # Password reset requests.
    pending_resets = (
        AuthController.get_pending_resets()
    )
# =====
# SEND DATA TO dashboard.html
# =====

```

```

return render_template(
    "dashboard.html",

    items=items,

    categories=categories,

    search=search,

    selected_category=category,

    total_stocks=total_stocks,

    active_loans=active_loans,

    history=history,

    pending_returns=pending_returns,

    pending_borrows=pending_borrows,

    pending_borrow_requests=(
        pending_borrow_requests
    ),

    all_loans=all_loans,

    pending_resets=pending_resets
)
# =====
# 11. BORROW ITEM
# =====
@app.route(
    "/borrow",
    methods=["POST"]
)
@login_required
def borrow():

    # Get item ID and quantity from the HTML form.
    try:

        item_id = int(
            request.form["item_id"]
        )

        quantity = int(
            request.form.get(
                "quantity",
                "1"
            )
        )

    except (KeyError, ValueError):

        flash(
            "Invalid item or borrow quantity.",
            "danger"
        )

        return redirect(

```

```

        url_for("dashboard")
    )

# Quantity must be at least 1.
if quantity < 1:

    flash(
        "Borrow quantity must be at least 1.",
        "danger"
    )

    return redirect(
        url_for("dashboard")
    )

# Only USER accounts can borrow.
if session["role"] != "USER":

    flash(
        "Only USER accounts can borrow equipment.",
        "danger"
    )

    return redirect(
        url_for("dashboard")
    )

# Use the existing controller.
ok, msg = InventoryController.borrow_item(
    session["username"],
    item_id,
    quantity
)

# Display the result.
flash(
    msg,
    "success" if ok else "danger"
)

return redirect(
    url_for("dashboard")
)

# =====
# 12. REQUEST RETURN
# =====
@app.route(
    "/return-request",
    methods=["POST"]
)
@login_required
def return_request():

    # Get selected loan IDs.
    raw_ids = request.form.getlist(
        "loan_ids"
    )

    try:

        loan_ids = [

```

```

        int(x)
        for x in raw_ids
    ]

except ValueError:

    loan_ids = []

# Only USER accounts can request returns.
if session["role"] != "USER":

    flash(
        "Only USER accounts can request returns.",
        "danger"
    )

    return redirect(
        url_for("dashboard")
    )

# Use the existing return function.
ok, msg = (
    InventoryController.request_bulk_item_returns(
        loan_ids
    )
)

flash(
    msg,
    "success" if ok else "warning"
)

return redirect(
    url_for("dashboard")
)
# =====
# 13. CHANGE PASSWORD
# =====
@app.route(
    "/change-password",
    methods=["POST"]
)
@login_required
def change_password():

    # Get current and new passwords.
    old_password = request.form.get(
        "old_password",
        ""
    )

    new_password = request.form.get(
        "new_password",
        ""
    )

    # Both fields are required.
    if not old_password or not new_password:

        flash(
            "Both current and new passwords are required.",

```

```

        "danger"
    )

    return redirect(
        url_for("dashboard")
    )

# Use the existing password function.
ok, msg = AuthController.change_password_direct(
    session["username"],
    session.get("email", ""),
    old_password,
    new_password
)

flash(
    msg,
    "success" if ok else "danger"
)

return redirect(
    url_for("dashboard")
)
# =====
# 14. ADMIN - ADD HARDWARE
# =====
@app.route(
    "/admin/add",
    methods=["POST"]
)
@admin_required
def admin_add():

    # Get hardware information.
    try:

        name = request.form.get(
            "item_name",
            ""
        ).strip()

        category = request.form.get(
            "category",
            ""
        ).strip()

        quantity = int(
            request.form.get(
                "quantity",
                ""
            )
        )

        unit_price = float(
            request.form.get(
                "unit_price",
                ""
            )
        )

    except ValueError:

```

```

    flash(
        "Quantity must be an integer and unit price must be numeric.",
        "danger"
    )

    return redirect(
        url_for("dashboard")
    )

# Use the original add_item function.
ok, msg = InventoryController.add_item(
    name,
    category,
    quantity,
    unit_price
)

flash(
    msg,
    "success" if ok else "danger"
)

return redirect(
    url_for("dashboard")
)
# =====
# 15. ADMIN - DELETE HARDWARE
# =====
@app.route(
    "/admin/delete",
    methods=["POST"]
)
@admin_required
def admin_delete():

    # Get selected hardware IDs.
    raw_ids = request.form.getlist(
        "item_ids"
    )

    try:

        item_ids = [
            int(x)
            for x in raw_ids
        ]

    except ValueError:

        item_ids = []

    # Delete selected items.
    ok, msg = InventoryController.delete_bulk_items(
        item_ids
    )

    flash(
        msg,
        "success" if ok else "warning"
    )

```

```

        return redirect(
            url_for("dashboard")
        )
# =====
# 16. ADMIN - BORROW APPROVAL
# =====
@app.route(
    "/admin/borrow-action",
    methods=["POST"]
)
@admin_required
def admin_borrow_action():

    # Get selected borrow request IDs.
    raw_ids = request.form.getlist(
        "loan_ids"
    )

    # Approve when the form action is "approve".
    approve = (
        request.form.get("action")
        == "approve"
    )

    try:

        loan_ids = [
            int(x)
            for x in raw_ids
        ]

    except ValueError:

        loan_ids = []

    # Process the borrow requests.
    ok, msg = (
        InventoryController.process_bulk_borrows(
            loan_ids,
            approve=approve
        )
    )

    flash(
        msg,
        "success" if ok else "danger"
    )

    return redirect(
        url_for("dashboard")
    )
# =====
# 17. ADMIN - RETURN APPROVAL
# =====
@app.route(
    "/admin/return-action",
    methods=["POST"]
)
@admin_required
def admin_return_action():

```

```

# Get selected return request IDs.
raw_ids = request.form.getlist(
    "loan_ids"
)

# Check whether Admin approved the return.
approve = (
    request.form.get("action")
    == "approve"
)

try:

    loan_ids = [
        int(x)
        for x in raw_ids
    ]

except ValueError:

    loan_ids = []

# Process return requests.
ok, msg = (
    InventoryController.process_bulk_returns(
        loan_ids,
        approve=approve
    )
)

flash(
    msg,
    "success" if ok else "danger"
)

return redirect(
    url_for("dashboard")
)
# =====
# 18. ADMIN - PASSWORD RESET APPROVAL
# =====
@app.route(
    "/admin/reset-action",
    methods=["POST"]
)
@admin_required
def admin_reset_action():

    # Get selected reset request IDs.
    raw_ids = request.form.getlist(
        "request_ids"
    )

    # Check whether Admin approved the reset.
    approve = (
        request.form.get("action")
        == "approve"
    )

    try:

```

```

        request_ids = [
            int(x)
            for x in raw_ids
        ]

except ValueError:

    request_ids = []

# Process reset requests.
ok, msg = (
    AuthController.process_bulk_resets(
        request_ids,
        approve=approve
    )
)

flash(
    msg,
    "success" if ok else "danger"
)

return redirect(
    url_for("dashboard")
)
# =====
# 19. EXPORT INVENTORY REPORT
# =====
@app.route("/export")
@login_required
def export():

    # Create the CSV report using the existing controller.
    ok, msg = InventoryController.export_to_csv(
        session["username"]
    )

    # If export failed, return to dashboard.
    if not ok:

        flash(
            msg,
            "danger"
        )

        return redirect(
            url_for("dashboard")
        )

    # Get the CSV file path.
    path = os.path.abspath(
        "inventory_report.csv"
    )

    # Make sure the file exists.
    if not os.path.exists(path):

        flash(
            "The CSV report could not be found.",
            "danger"

```

```

    )

    return redirect(
        url_for("dashboard")
    )

# Send the CSV file to the browser.
return send_file(
    path,
    as_attachment=True,
    download_name="inventory_report.csv"
)
# =====
# 20. LOGOUT
# =====
@app.route("/logout")
def logout():

    # Remove the user's session.
    session.clear()

    flash(
        "You have been logged out.",
        "success"
    )

    return redirect(
        url_for("login")
    )
# =====
# 21. START THE FLASK WEB SERVER
# =====
if __name__ == "__main__":

    # Initialize the existing database.
    init_db()

    print()
    print("=" * 58)
    print(" CAMPUS HARDWARE INVENTORY - WEB PORTAL")
    print("=" * 58)
    print()

    print(" Open Google Chrome and go to:")
    print(" http://127.0.0.1:5000")

    print()
    print(" Press CTRL+C to stop the server.")
    print("=" * 58)
    print()

    # Start Flask.
    app.run(debug=True)

```

Step 9 — Create/Use html

Use base.html, login.html, register.html, reset.html and dashboard.html. Preserve the layout, cards, tables, navigation, responsive design and action controls. Do not move database/controller logic into HTML.

Step 10 — Implement the Current Dashboard Requirements

Use the dashboard requirements in Section 7. Make sure the correct dashboard is shown for ADMIN versus USER. Do not display Total Asset Valuation.

Step 11 — Implement Quantity-Based Borrowing

Add a quantity input beside the borrow action. Minimum = 1. Maximum = available stock. The student selects an equipment item and enters the quantity requested.

Step 12 — Implement Borrow Approval

A borrow request must first become PENDING_BORROW. Do not deduct stock while it is pending. Admin reviews the requested item and quantity. Approve changes it to BORROWED and deducts the approved quantity. Reject does not create a borrowed item.

Step 13 — Keep Return Approval

A borrowed item can be submitted for return. The request becomes RETURN_PENDING. Admin approves to restock the returned quantity and mark it RETURNED, or rejects to keep it BORROWED.

Step 14 — Test Authentication

Test valid login, invalid login and the three-failed-attempt logout. Do not duplicate password rules in JavaScript.

Step 15 — Test Inventory and Dashboard

Verify that inventory records display correctly, Total Stocks is visible, search/filter works, and role-specific cards/sections appear.

Step 16 — Test Borrowing with Quantity

Use an item with at least several available units. Request 2 or 3 units. Confirm that the request is pending and stock is not reduced until Admin approval.

Step 17 — Test Admin Approval

Log in as ADMIN. Review Pending Borrow Requests. Approve one request and verify that the stock decreases by the requested quantity and the user's Borrowed Items shows the same quantity.

Step 18 — Test Return Approval

As USER, request a return. As ADMIN, approve it and verify that the quantity is added back to inventory. Test rejection separately and verify the item remains borrowed.

Step 19 — Test History and Security

Check My Borrow History, Borrow Audit, password reset workflow and logout. Verify protected pages require login.

Step 20 — Run and Document the Web Application

Start the Flask application using python app.py. Open Google Chrome and go to http://127.0.0.1:5000. Capture screenshots of the working application and required workflows.

7. LATEST DASHBOARD REQUIREMENTS

Use the web app UI design, but apply the following current requirements. This table is the student's checklist.

Dashboard Element	ADMIN	USER
Total Stocks	YES	YES
Account Role	YES	YES
Borrowed Items	YES	YES
Pending Borrow Requests	YES	YES

Pending Returns	YES / admin approval panel	Not an admin approval panel; show the user's relevant pending return information if implemented in the supplied page
Password Reset Requests	YES / admin approval panel	No admin approval panel
Borrow Audit	YES	No
Total Asset Valuation	NO	NO
Visible Items	NO — use Total Stocks	NO — use Total Stocks
Active Loans wording	NO — use Borrowed Items where visible	NO — use My Borrowed Items

Important: Total Stocks means the total quantity of inventory stocks, not merely the number of rows/items visible after a search. If the current app.py passes the total separately, use that variable. Do not invent a new controller method just to change the label.

8. LATEST BORROWING WORKFLOW WITH QUANTITY

Stage	USER	ADMIN	Inventory Stock
1. Select equipment	Select item and enter quantity	—	No change
2. Request Borrow	Submit request	—	No change
3. Pending	Can see pending borrow request	Can review requested item + quantity	No change
4. Approve	—	Approve Borrow	Decrease by approved quantity
5. Borrowed	Can see Borrowed Items and quantity	Can see audit	Quantity remains deducted
6. Request Return	Request return	—	No change
7. Return Pending	Can see pending return	Can review	No change
8. Approve Return	—	Approve & Restock	Increase by returned quantity
9. Returned	History records return	History/audit available	Quantity restored
10. Reject Return	—	Reject Return	No change; remains borrowed

Example: If an Arduino Uno has 10 available units and the student requests 3, the system should remain at 10 while the request is pending. After Admin approval, it should become 7. After an approved return of all 3 units, it should become 10 again.

9. Quantity Rules

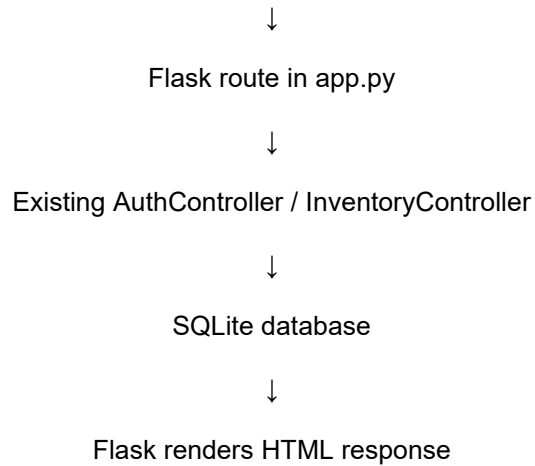
- Quantity must be a whole number.
- Quantity must be at least 1.
- Quantity must not exceed the currently available stock.
- A pending borrow must not reduce stock.
- An approved borrow reduces stock by the approved quantity.
- A rejected borrow must not reduce stock.
- An approved return increases stock by the returned quantity.
- A rejected return leaves the borrowed quantity unchanged.
- Existing older loan records may be treated as quantity 1 when a quantity field is added through a migration.

10. Flask Bridge Concept

The browser does not directly call InventoryController. The browser submits a form to Flask. Flask reads the form, calls the original Python controller, then renders or redirects to an HTML page.

Conceptual flow:

Browser



Keep HTML responsible for structure/content, Python for processing/business logic, and JavaScript for small browser interactions.

11. Running the Application

From PowerShell in the web project folder:

```
conda activate gender_env    (if your class is using the existing Conda environment)
```

```
python -m pip install -r requirements.txt
```

```
python app.py
```

The application should print a clear portal message similar to:

```
=====
CAMPUS HARDWARE INVENTORY - WEB PORTAL
=====

Open Google Chrome and go to:
http://127.0.0.1:5000

Press CTRL+C to stop the server.
=====
```

Open Google Chrome and enter: <http://127.0.0.1:5000>

12. Requirements

- Flask==3.1.2
- bcrypt==5.0.0
- pydantic>=2.0
- gunicorn==23.0.0

13. Testing Checklist

- ☐ 1. Application starts without a Python syntax error.
- ☐ 2. Chrome can open <http://127.0.0.1:5000>.

- ☐ 3. Login page uses the NICE UI.
- ☐ 4. Valid login works.
- ☐ 5. Invalid login displays an appropriate message.
- ☐ 6. Three failed attempts lock the account as expected.
- ☐ 7. Registration is on a separate page.
- ☐ 8. Reset/Unlock is on a separate page.
- ☐ 9. Hardware Catalog displays records.
- ☐ 10. Search/filter works if included.
- ☐ 11. Total Stocks displays a number.
- ☐ 12. Total Asset Valuation is not displayed.
- ☐ 13. User sees Borrowed Items / My Borrowed Items.
- ☐ 14. Admin sees Pending Borrow Requests.
- ☐ 15. User can enter a borrow quantity.
- ☐ 16. Quantity cannot exceed available stock.
- ☐ 17. Pending borrow does not reduce stock.
- ☐ 18. Admin can approve a borrow.
- ☐ 19. Approved borrow reduces stock by the requested quantity.
- ☐ 20. Admin can reject a borrow.
- ☐ 21. User can request a return.
- ☐ 22. Pending return does not immediately restock.
- ☐ 23. Admin can approve a return and stock is restored.
- ☐ 24. Admin can reject a return and the item remains borrowed.
- ☐ 25. Borrowed Items displays the quantity.
- ☐ 26. History/audit displays the quantity where applicable.
- ☐ 27. Logout works.
- ☐ 28. Protected pages cannot be opened after logout.

14. Desktop vs Web Comparison

Feature	Original Desktop	Web Version	What Changed? / What Stayed?
Login	Tkinter + Python	HTML + Flask + original AuthController	Interface changed; authentication logic reused.
Registration	Tkinter + Python	HTML + Flask + original AuthController	Interface changed; controller reused.
Password Reset	Tkinter + Python	HTML + Flask + original AuthController	Interface changed; reset logic reused.
Hardware Catalog	Tkinter Treeview	HTML table + Flask	Display changed; inventory data/controller reused.
Borrowing	Desktop button/action	HTML form + Flask	Workflow updated to approval and quantity.
Return	Desktop action	HTML form + Flask	Approval workflow retained/used.
Admin Approval	Desktop GUI	Web admin dashboard	Interface changed; approval logic reused/extended.
History	Tkinter Treeview	HTML table	Display changed; data logic reused.
Database	SQLite	SQLite	NO CHANGE for Lab 1.
Desktop GUI	Tkinter	Browser UI	Tkinter is retained but not called by Flask.

15. Reflection Questions

1. Why can AuthController and InventoryController be reused even though the GUI changes?

Because the controllers already have the main functions of the system. We only changed the GUI from Tkinter to a web interface, so we can still use the existing controllers.

2. Why can Tkinter widgets not simply appear in a browser?

Because Tkinter is used for desktop applications. A browser uses HTML, CSS, and JavaScript, so Tkinter widgets cannot directly work or appear in the browser.

3. What is the role of app.py?

app.py acts as the bridge between the browser and the Python system. It receives the request from the browser and calls the existing Python controllers.

4. Why should the original desktop source be preserved?

We should preserve it because it is already working and contains the original database and system logic. It can also be used as a backup and reference for the web version.

5. What is the difference between reusing code and copying/rebuilding code?

Reusing code means using the existing functions and controllers in the new system. Rebuilding means creating the same functions again from the beginning. Reusing saves time and follows the minimum-change approach.

6. How does an HTML form reach a Python function?

The HTML form sends the information to a Flask route in app.py. Flask gets the data and calls the needed Python function or controller.

7. Why should business rules remain in Python instead of JavaScript?

Because Python should handle the main processing and business rules. JavaScript is only used for small browser interactions and should not duplicate the main system logic.

8. Why does a pending borrow not immediately reduce stock?

Because the borrow request is still waiting for Admin approval. The stock will only decrease once the Admin approves the request.

9. Why must the requested quantity be checked against available stock?

So the user cannot borrow more items than what is available. The quantity must be at least 1 and must not be higher than the available stock.

10. What is the difference between Borrowed Items and Pending Borrow Requests?

Borrowed Items are items that have already been approved and borrowed. Pending Borrow Requests are requests that are still waiting for Admin approval.

11. Why is SQLite kept for this local laboratory?

SQLite is kept because this laboratory is for a local web application. It is simple to use and allows the desktop and web versions to use the same local database.

12. What would need to change before the system is published publicly?

The local SQLite database should be changed to a persistent, hosted database, such as PostgreSQL or Supabase. The Flask app would also need to be hosted so users can access it through a public HTTPS URL.

16. Required Deliverables

- Original desktop Python source preserved.
- Backup copy of hardware_inventory.db.
- New/updated app.py Flask bridge.
- templates/ containing the required HTML pages.
- static/ containing CSS and any required JavaScript.
- requirements.txt.
- Working local browser application screenshot.
- Screenshot showing a user borrow request with quantity.
- Screenshot showing Admin Pending Borrow Requests.
- Screenshot showing an approved borrow and updated stock.
- Screenshot showing return approval/restocking.
- Completed Desktop vs Web comparison table.
- Short reflection answering the questions.

17. Student Warning — Do Not Make Unnecessary Changes

DO NOT rewrite the whole Python application just because the interface is changing.

DO NOT delete your working Tkinter code unless your instructor specifically tells you to.

DO NOT move database operations into HTML.

DO NOT duplicate authentication or business rules in JavaScript.

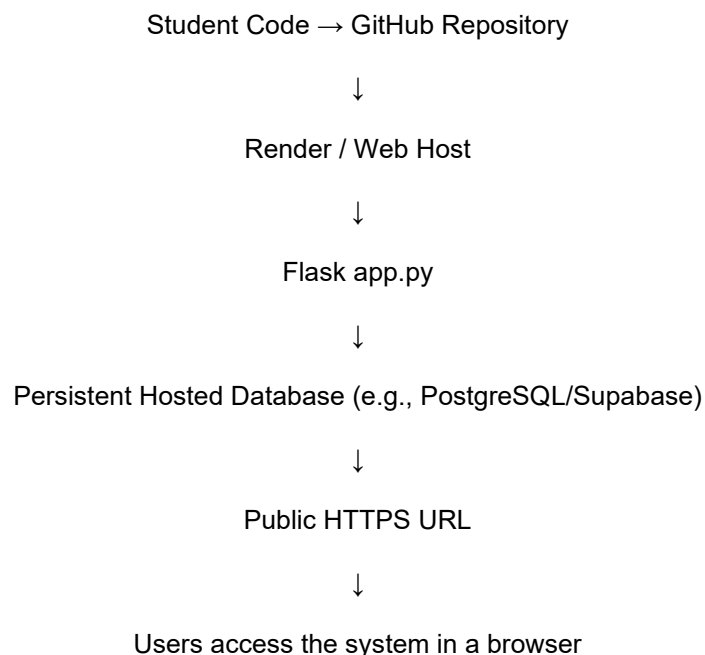
DO add only the web bridge, web templates, styling and the specifically required borrow/quantity workflow changes.

18. One-Minute Explanation for the Student

“Our original system already had the database and Python functions for authentication, inventory, borrowing, returns and administration. We did not throw those away. We added Flask so the browser can send requests to those existing Python functions. We replaced the Tkinter screens with HTML pages. CSS controls the appearance, and JavaScript handles small browser interactions. The latest requirement adds quantity-based borrowing and an Admin approval step, so stock is changed only after approval. Therefore, the conversion is mainly a change in interface and communication method, not a complete rewrite of the system.”

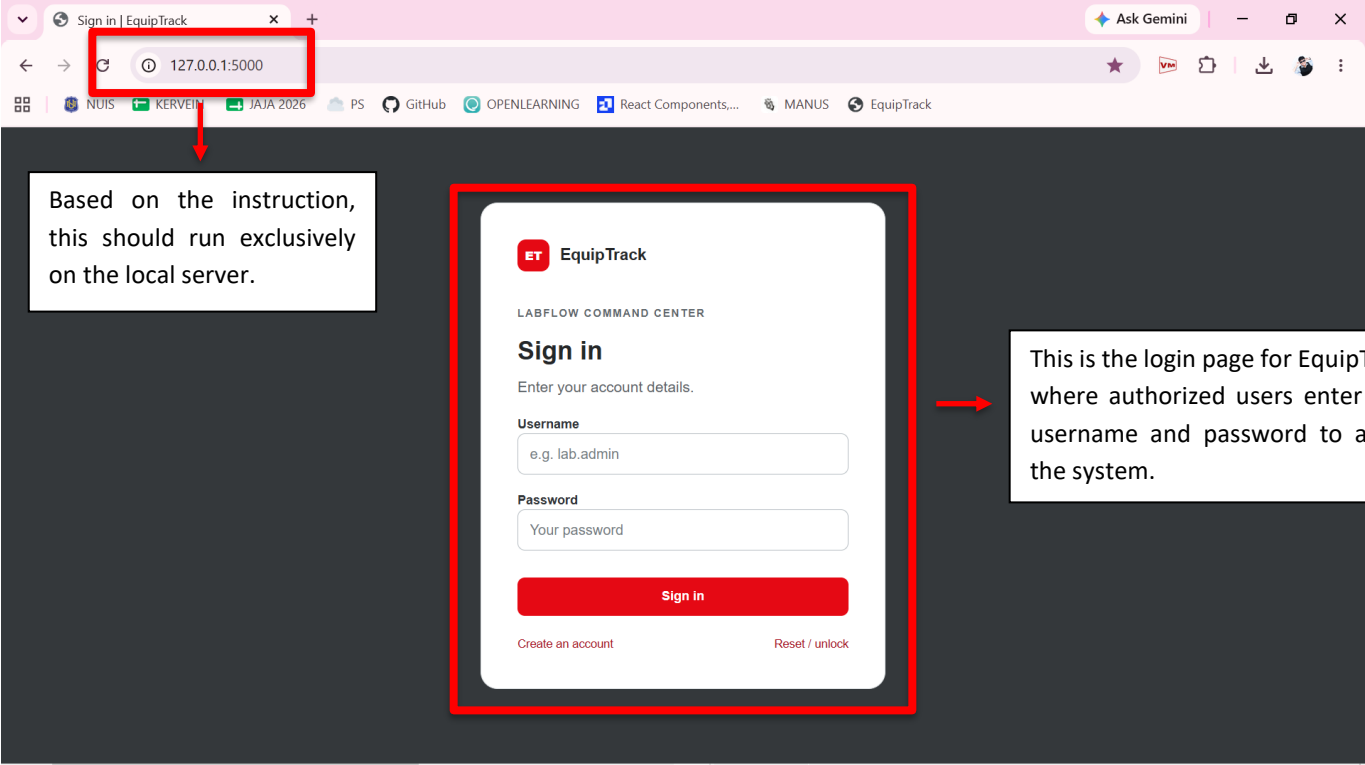
19. Online Deployment Preview

Only after the local web version works should the project proceed to GitHub and public hosting. A typical later architecture is:



For this laboratory, SQLite is intentionally retained so students can see that the same database can support the desktop and local web interfaces. For public deployment, use a persistent hosted database rather than relying on local server storage.

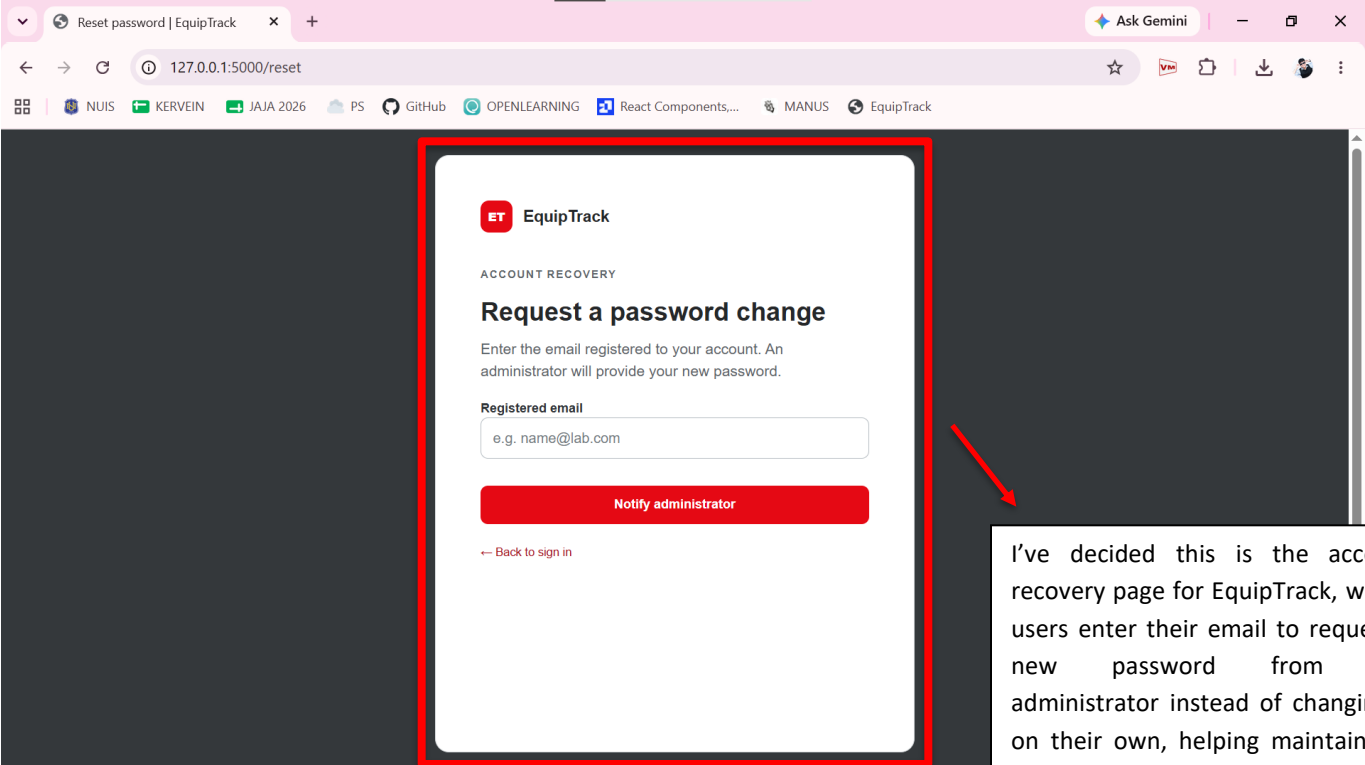
EQUIPTRACK WEBSITE



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000". The page title is "Sign in | EquipTrack". The main content area features a white card with the EquipTrack logo and the text "LABFLOW COMMAND CENTER". Below this is a "Sign in" heading followed by the instruction "Enter your account details." There are two input fields: "Username" with the placeholder "e.g. lab.admin" and "Password" with the placeholder "Your password". A red "Sign in" button is positioned below the fields. At the bottom of the card, there are links for "Create an account" and "Reset / unlock".

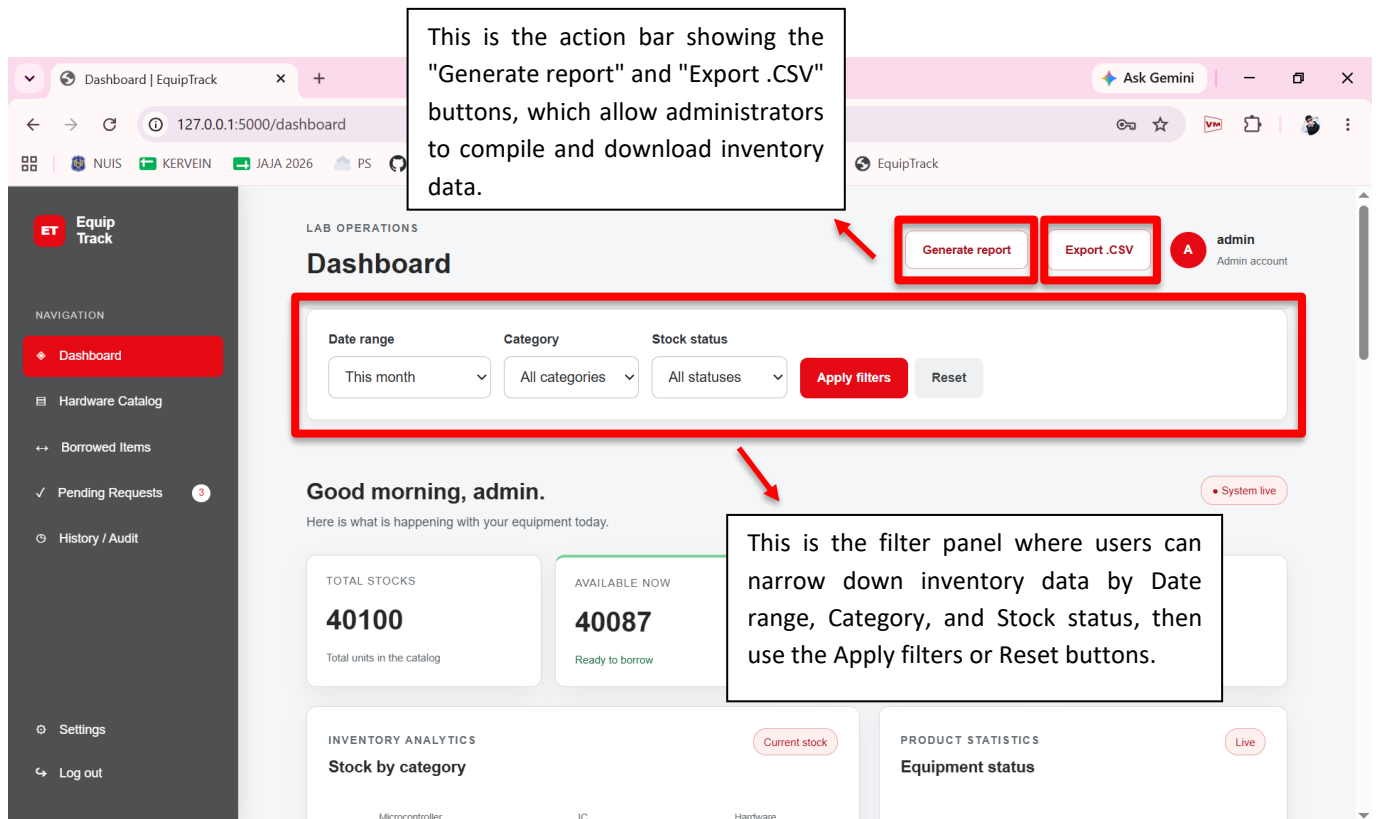
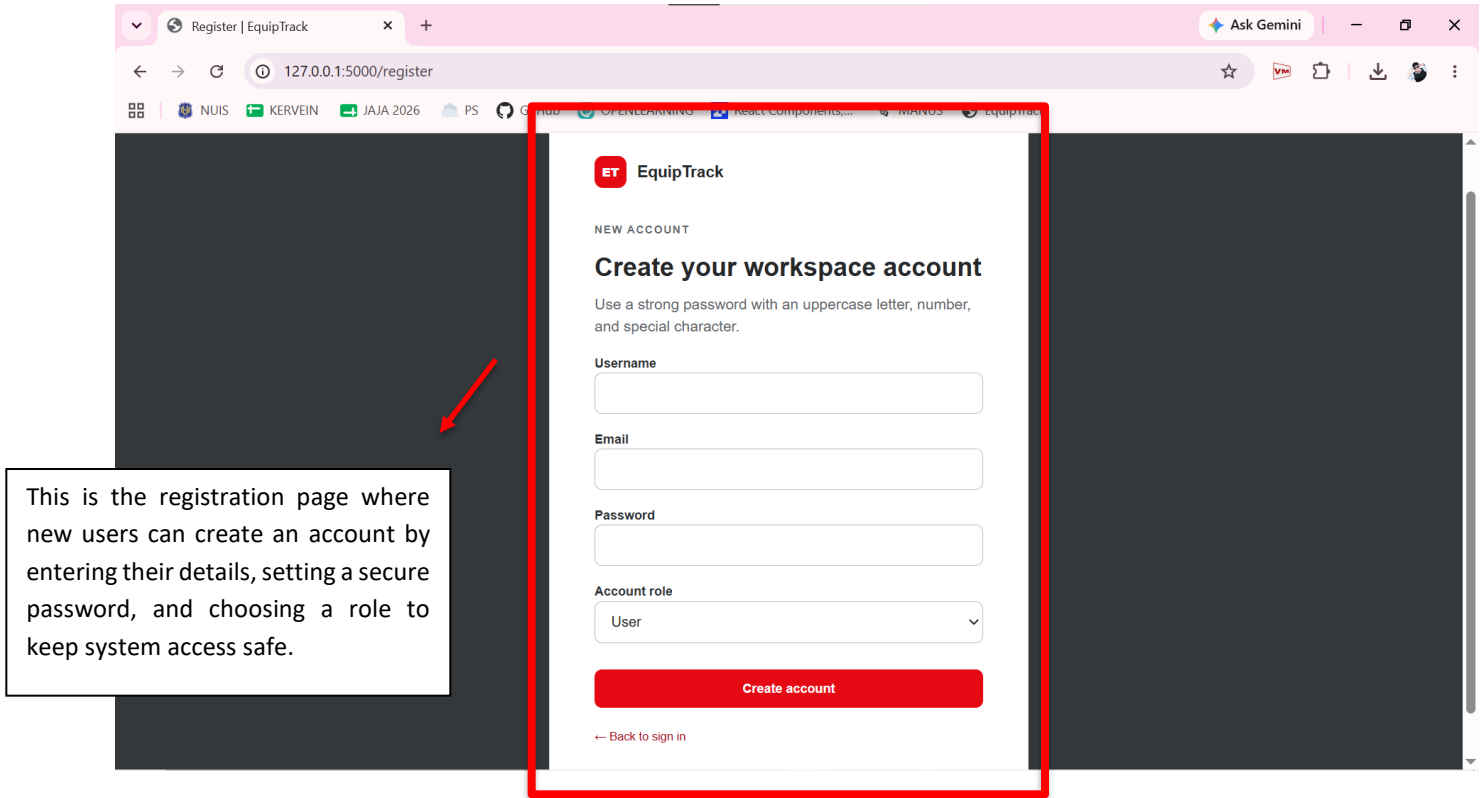
Based on the instruction, this should run exclusively on the local server.

This is the login page for EquipTrack, where authorized users enter their username and password to access the system.



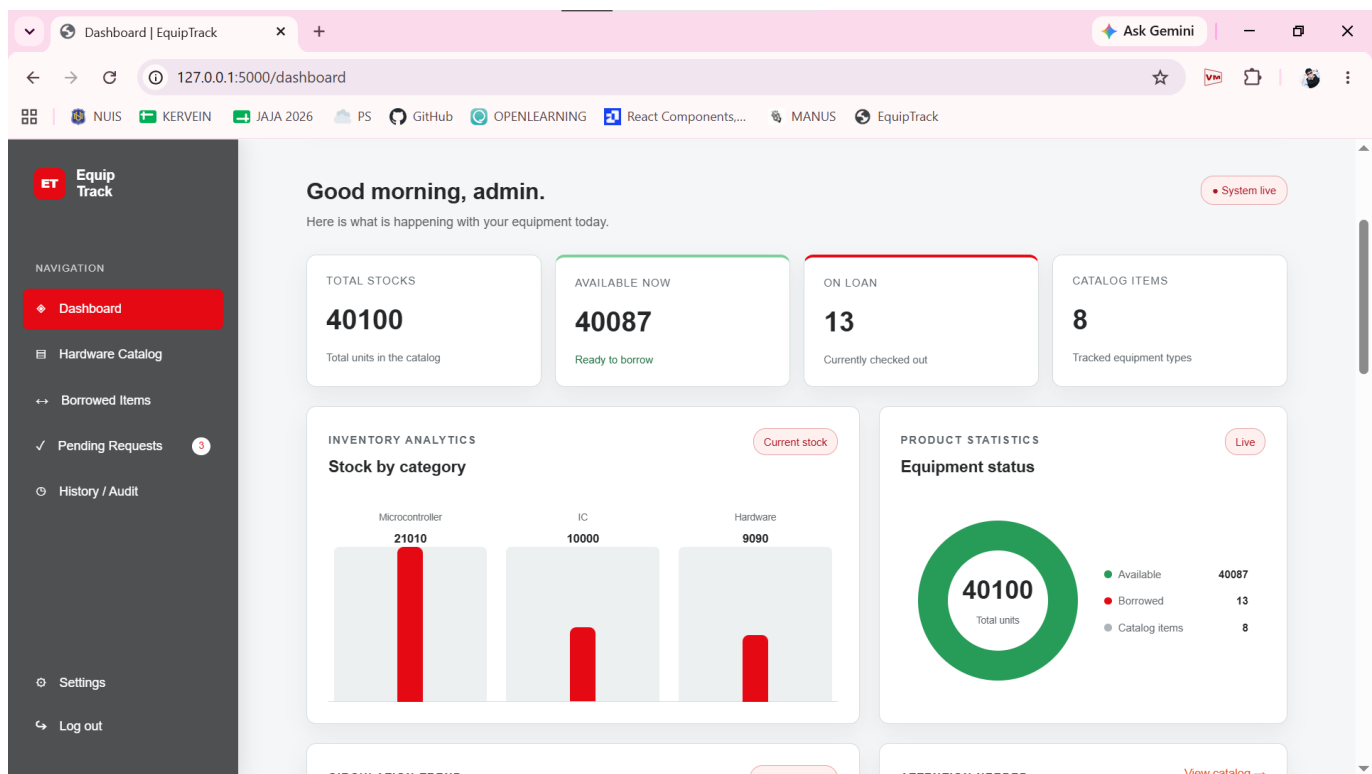
The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/reset". The page title is "Reset password | EquipTrack". The main content area features a white card with the EquipTrack logo and the text "ACCOUNT RECOVERY". Below this is a "Request a password change" heading followed by the instruction "Enter the email registered to your account. An administrator will provide your new password." There is one input field: "Registered email" with the placeholder "e.g. name@lab.com". A red "Notify administrator" button is positioned below the field. At the bottom of the card, there is a link for "← Back to sign in".

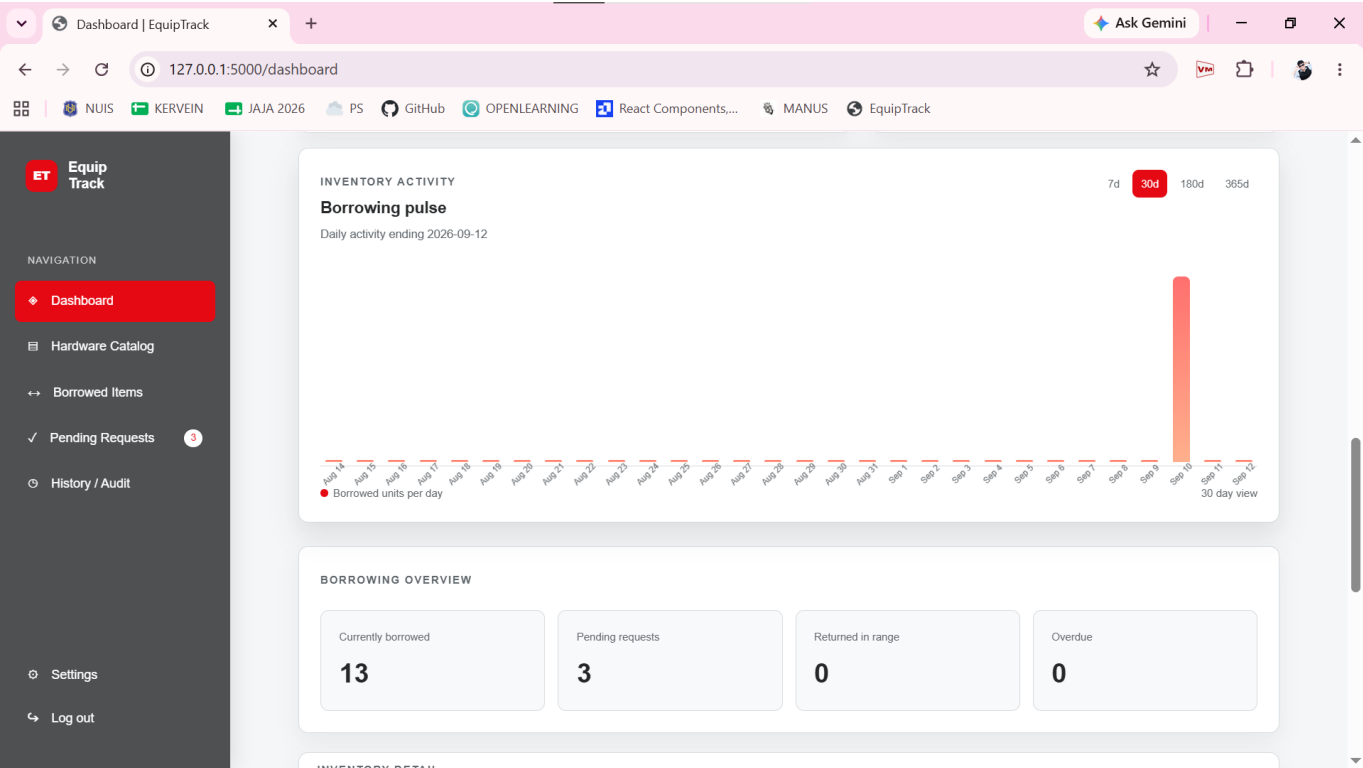
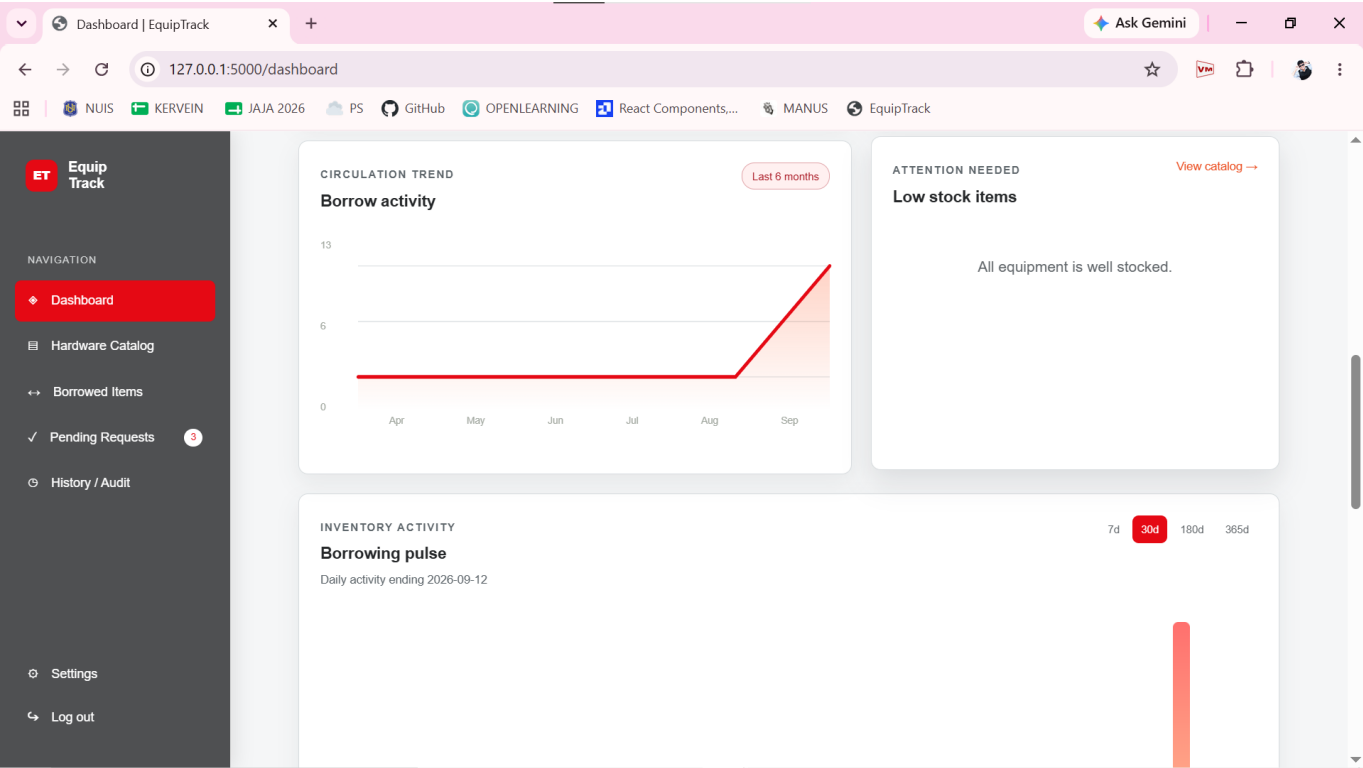
I've decided this is the account recovery page for EquipTrack, where users enter their email to request a new password from the administrator instead of changing it on their own, helping maintain the system security by ensuring administrative oversight and preventing unauthorized access.



The EquipTrack dashboard provides a complete overview of lab inventory and equipment circulation:

- **Top Statistics:** Displays total stock units (40,100), available items (40,087), active loans (13), and tracked catalog equipment types (8).
- **Inventory Analytics & Status:** Visualizes stock breakdown by categories (Microcontroller, IC, Hardware) and shows overall equipment status distribution.
- **Circulation Trends & Pulse:** Tracks borrow activity over time with interactive time filters (7d, 30d, 180d, 365d) alongside detailed borrowing overview metrics (currently borrowed, pending requests, returns, and overdue items).
- **Equipment Register:** Lists individual hardware items—such as 74HC00, Arduino NANO, and ESP32—with their respective categories, available counts, borrowed amounts, and stock status.





Dashboard | EquipTrack

127.0.0.1:5000/dashboard

Ask Gemini

NAVIGATION

- Dashboard
- Hardware Catalog
- Borrowed Items
- Pending Requests 3
- History / Audit
- Settings
- Log out

INVENTORY DETAIL

Equipment register

Equipment	Category	Available	Borrowed	Status
74HC00	IC	9000	0	In Stock
74HC08	IC	1000	0	In Stock
Arduino ATMEGA	Microcontroller	10	0	In Stock
Arduino NANO	Microcontroller	9987	13	In Stock
Arduino UNO	Microcontroller	1000	0	In Stock
Breadboard	Hardware	1000	0	In Stock
ESP32	Microcontroller	10000	0	In Stock
LED	Hardware	8090	0	In Stock

Hardware Catalog | EquipTrack

127.0.0.1:5000/catalog

Ask Gemini

NAVIGATION

- Dashboard
- Hardware Catalog
- Borrowed Items
- Pending Requests
- History / Audit
- Settings
- Log out

EQUIPMENT LIBRARY

Hardware Catalog

admin
Admin account

Browse equipment and request the quantity you need

ADMIN TOOLS

Add equipment item

Item name Category Quantity Unit price

Serial number Condition

Equipment	Category	Available	Condition	Status	Borrowed	Manage
Arduino UNO	Microcontroller	1000	Good	In Stock	0	Edit Delete
Arduino NANO	K12345					Edit Delete

This is the **Hardware Catalog** page, where administrators can view equipment, manage existing inventory items using Edit or Delete options, and use the Add equipment item form to input details like item name, category, quantity, unit price, serial number, and condition.

Hardware Catalog | EquipTrack

127.0.0.1:5000/catalog

NAVIGATION

- Dashboard
- Hardware Catalog
- Borrowed Items
- Pending Requests
- History / Audit
- Settings
- Log out

Equipment	Category	Available	Condition	Status	Borrowed	Manage
Arduino UNO None	Microcontroller	1000	Good	In Stock	0	Edit Delete
Arduino NANO K12345	Microcontroller	9987	Good	In Stock	13	Edit Delete
74HC00 K123456	IC	9000	Good	In Stock	0	Edit Delete
74HC08 K010101	IC	1000	Needs Inspection	In Stock	0	Edit Delete
Breadboard K09878	Hardware	1000	Excellent	In Stock	0	Edit Delete
LED K0987	Hardware	8090	Excellent	In Stock	0	Edit Delete
ESP32 K9292	Microcontroller	10000	Excellent	In Stock	0	Edit Delete
Arduino ATMEGA K09090	Microcontroller	10	Excellent	In Stock	0	Edit Delete

Borrowed Items | EquipTrack

127.0.0.1:5000/borrowed

NAVIGATION

- Dashboard
- Hardware Catalog
- Borrowed Items
- Pending Requests
- History / Audit
- Settings
- Log out

Borrowed Items

admin
Admin account

View active loans and submit return requests

Equipment	Borrower	Student ID	Quantity	Borrowed at	Return
Arduino NANO	koko	2023-100141	13	2026-09-10 19:39:31	13 Request return

This section displays active loans in a table format, tracking equipment names, borrower details, student IDs, quantities, checkout timestamps, and providing an action field to request a return.

This is the **administration page for pending requests**, where administrators can review, approve, or reject incoming borrow and return requests from users.

ADMINISTRATION

Pending Requests

Approve or reject borrowing and return requests.

Borrow requests

Item	Requester	Student ID	Quantity	Action
74HC00	koko	2023-100141	200	<button>Approve</button> <button>Reject</button>
LED	koko	2023-100141	57	<button>Approve</button> <button>Reject</button>

Return requests

Item	Requester	Quantity	Action
Arduino NANO	koko	12	<button>Approve</button> <button>Reject</button>

AUDIT TRAIL

History / Audit

Administrative activity and equipment history.

Time	User	Action	Details
2026-09-11 19:54:39	admin	LOGIN	Successful login
2026-09-11 19:49:24	admin	LOGIN	Successful login
2026-09-11 19:47:51	admin	LOGIN	Successful login
2026-09-11 19:46:39	admin	EQUIPMENT_UPDATED	Item updated successfully.
2026-09-11 19:45:41	admin	EQUIPMENT_UPDATED	Item updated successfully.
2026-09-11 19:45:27	admin	EQUIPMENT_UPDATED	Item updated successfully.
2026-09-11 19:45:01	admin	LOGIN	Successful login

This is the **History / Audit page**, where administrators can track system logs, user actions, timestamps, and details such as logins and equipment updates.

The screenshot shows the EquipTrack Settings page. The left sidebar contains a navigation menu with options: Dashboard, Hardware Catalog, Borrowed Items, Pending Requests, and History / Audit. The main content area is titled 'Settings' and includes a sub-header 'PREFERENCES'. Below this, there is a section for 'APPEARANCE' with a 'Theme' dropdown set to 'Light mode' and a 'Dark mode' toggle switch. A red box highlights the 'APPEARANCE' section. A red arrow points from the 'Dark mode' toggle to a text box.

Settings

Customize how EquipTrack looks and feels on this device.

APPEARANCE

Theme

Switch between a bright workspace and a low-light interface.

Dark mode

Use darker surfaces and softer contrast.

This is the **settings page**, where users can customize appearance preferences, switch between themes, and toggle dark mode for a low-light interface.

The screenshot shows the EquipTrack Settings page, specifically the 'ACCOUNT SECURITY' section. The left sidebar is the same as in the first image. The main content area is titled 'Account security' and includes a sub-header 'Change your password'. Below this, there are input fields for 'Current password', 'New password', and 'Confirm new password', along with a 'Change admin password' button. Another section, 'Unlock or reset an account', includes a dropdown for 'Account' and a 'New password' input field, with an 'Unlock and reset account' button. A red box highlights the 'ACCOUNT SECURITY' section. A red arrow points from the 'Unlock and reset account' button to a text box.

ACCOUNT SECURITY

Account security

Manage your administrator password and reset or unlock user accounts.

Change your password

Update the administrator password directly using your current password.

Current password **New password** **Confirm new password**

Change admin password

Unlock or reset an account

Reset a user's password and clear the lockout after repeated failed sign-in attempts.

No accounts are currently locked.

Account **New password**

Select an account Use uppercase, numb

Unlock and reset account

This section of the **settings page** provides account security controls, allowing administrators to update passwords and unlock or reset user accounts.

Hardware Catalog | EquipTrack

127.0.0.1:5000/catalog

Ask Gemini

NAVIGATION

- Hardware Catalog
- My Borrowing

Settings

Log out

EQUIPMENT LIBRARY

Hardware Catalog

koko User account

Browse equipment and request the quantity you need.

USER REQUEST

What would you like to borrow?

Enter your item, student ID, and requested quantity.

Equipment name:

Quantity:

Student ID:

Request borrow

Equipment	Category	Available	Condition	Status
Arduino UNO None	Microcontroller	1000	Good	In Stock
Arduino NANO K12345	Microcontroller	9987	Good	In Stock
74HC00	IC	9000	Good	In Stock

Hardware Catalog & Request: Browse lab items, check stock, and request equipment by entering your name, student ID, and quantity.

My Borrowing | EquipTrack

127.0.0.1:5000/my-borrowing

Ask Gemini

NAVIGATION

- Hardware Catalog
- My Borrowing

Settings

MY ACTIVITY

My Borrowing

koko User account

Track your borrow request decisions and return active equipment.

REQUEST NOTIFICATIONS 2 updates

Borrow request updates

- Breadboard**
12 unit(s) - Reviewed 2026-09-10 19:39:36
Rejected
- Arduino NANO**
13 unit(s) - Reviewed 2026-09-10 19:39:31
Approved

ACTIVE LOANS 1 active

Items not yet returned

- Arduino NANO**
13 unit(s) - Borrowed 2026-09-10 19:39:31 - Due 2026-09-24 19:39:31
Return pending

Request Notifications: Check the status of your borrow requests, such as approved or rejected items with timestamps.

Active Loans: View unreturned equipment, loan dates, due dates, and return statuses.

Feature / Area	Administrator Access (example: ME)	Student/User Access
Dashboard & System Control	I have full access to overall system analytics, inventory records, and complete activity history logs.	Students/User do not have access to management dashboards or internal history logs.
Hardware Catalog	I can add, edit, or delete items and inspect hardware conditions in the catalog.	Students/User can only browse available items and submit new borrow requests.
Request Processing	I hold the authority to review, approve, or reject incoming borrow and return requests.	Students/User can only submit their own requests and track their personal approval statuses.
Settings & Security	I manage system settings, passwords, and account unlocks.	Student/User access is strictly limited to personal borrowing views and basic navigation

Here's a quick and direct explanation from my side on why **I restricted user access** compared to my administrator's account:

- **Inventory Security & Control:** I restricted student users so they cannot accidentally modify, delete, or tamper with the core inventory records and item catalog that I manage.
- **Workflow Integrity:** I kept approval authority on my admin side so that I have complete control over verifying and reviewing who borrows or returns equipment.
- **System Privacy:** I blocked students from viewing internal system analytics, activity audit logs, and security settings to keep administrative tools and user management private.

My **administrator account** has access to manage and monitor users for the following reasons:

Account Security & Recovery: I gave my admin account access to the settings so I can securely change passwords and unlock or reset student accounts whenever they get locked out due to failed login attempts.

Oversight and Accountability: I kept admin access so I can monitor, review, and keep track of all student borrowing and returning requests in the system.